22

エージェント型コーディング評価におけるインフラノイズの定量化

— Quantifying infrastructure noise in agentic coding evals —

公開日	2026-04-09
原題	Quantifying infrastructure noise in agentic coding evals
著者	Anthropic Engineering Team
原文	https://www.anthropic.com/engineering/infrastructure-noise
翻訳	Claude（機械翻訳／Anthropic）
編集	2026-04-09

エージェント型コーディング評価におけるインフラノイズの定量化

SWE-bench や Terminal-Bench のようなエージェント型コーディングベンチマークは、フロンティアモデルのソフトウェアエンジニアリング能力を比較するために広く使われています——リーダーボード上位のスコアはわずか数パーセントポイントの差で競われることが多いです。これらのスコアはモデルの相対的な能力を正確に示す指標として扱われることが多く、どのモデルをデプロイするかという意思決定にも影響を与えるようになっています。しかし、インフラの設定だけでそのマージンを超える差が生じることを発見しました。社内実験では、Terminal-Bench 2.0 において最もリソースが多い設定と最も少ない設定の差が 6 パーセントポイント（$p < 0.01$）に達しました。

静的なベンチマークはモデルの出力を直接採点します——ランタイム環境は結果に影響しません。エージェント型コーディング評価は異なります。モデルはプログラムの作成、テストの実行、依存関係のインストール、複数ターンにわたる反復を行う完全な環境を与えられます。ランタイムはもはや受け身のコンテナではなく、問題解決プロセスの不可欠な構成要素です。リソース予算や時間制限が異なるエージェントたちは、同じテストを受けているとは言えません。

eval の開発者もこの点を考慮し始めています。Terminal-Bench 2.0 は例えば、最新の 2.0 リリースでタスクごとに推奨 CPU と RAM を指定しています。しかし、リソースを指定することと一貫してそれを強制することは同じではありません。さらに、強制の方法論が実際にベンチマークの測定対象を変えることも発見しました。

経緯

私たちは Terminal-Bench 2.0 を Google Kubernetes Engine クラスターで実行しています。セットアップの調整中に、スコアがベンチマークの公式リーダーボードと一致しないことや、インフラのエラー率が驚くほど高いことに気付きました。タスクの最大 6% がポッドエラーで失敗しており、その多くはモデルのタスク解決能力とは無関係でした。

スコアの差異は強制方法に起因していました。私たちの Kubernetes 実装は、タスクごとのリソース仕様をフロアかつハード上限として扱っていました。各コンテナは指定されたリソースが保証されていましたが、それを超えた瞬間に強制終了されます。コンテナランタイムは 2 つの別々のパラメーターでリソースを強制します。事前に確保する保証済み割り当てと、コンテナが強制終了されるハード上限です。これらが同じ値に設定されると、一時的なスパイクに対するヘッドルームがゼロになります。一瞬のメモリ変動が、それなければ成功してい

たはずのコンテナを OOM-kill してしまうのです。これに対処するため、Terminal-Bench のリーダーボードは異なるサンドボックスプロバイダーを使用しており、その実装はより寛容で、インフラの安定性を優先するためにコンテナを強制終了させずに一時的なオーバーアロケーションを許可しています。

この発見は、より大きな疑問を提起しました。リソース設定は評価スコアにどれほどの影響を与えるのか？

スキャフォールドの影響を定量化するため、Terminal-Bench 2.0 を 6 つのリソース設定で実行しました。タスク仕様の厳格な強制（1x）——フロアかつ上限として機能——から、完全な上限なしまでです。他のすべては一定に保ちました。同じ Claude モデル、同じハーネス、同じタスクセットです。

実験では、成功率はリソースのヘッドルームとともに増加しました。これは主にインフラのエラー率が各ステップで単調に低下したことで駆動されており、厳格な強制での 5.8% から上限なしでの 0.5% まで減少しました。厳格な強制から 3x ヘッドルームへの低下（5.8% から 2.1%）は $p < 0.001$ で有意でした。ヘッドルームが増えると、割り当てを超えたために強制終了されるコンテナが減ります。

1x から 3x の範囲では、成功スコアはノイズのマージン内で変動します（$p=0.40$）。1x でクラッシュしていたタスクの多くは、いずれにせよ失敗していたでしょう——これはデータでも確認されました。エージェントは探索し、リソースの壁にぶつかり、プリエンプトされますが、正解への道を歩んでいませんでした。

しかし 3x 前後からこのトレンドが変わります。成功率がインフラエラーの低下より速く上昇し始めるのです。

3x から上限なしにかけて、インフラエラーはさらに 1.6 パーセントポイント低下する一方、成功率は約 4 パーセントポイント跳ね上がります。追加のリソースにより、エージェントは大きな依存関係の取得、コストの高いサブプロセスの生成、メモリ集約的なテストスイートの実行など、潤沢な割り当てでしかうまくいかないアプローチを試みることができます。上限なしの場合、1x に対する合計の向上は +6 パーセントポイント（$p < 0.01$）です。マージンでは、`rstan-to-pystan` や `compile-compcert` のようなタスクが、メモリのヘッドルームを得ることで成功率を大幅に改善します。

これが測定に与える影響

Terminal-Bench 仕様の約 3x までは、追加リソースはインフラの信頼性の問題（一時的なリソーススパイク）を修正します。Terminal-Bench メンテナーが使用するサンドボックスプロバイダーは、暗黙的にこれを裏で行っており、eval をより難しくすることなく安定させています。

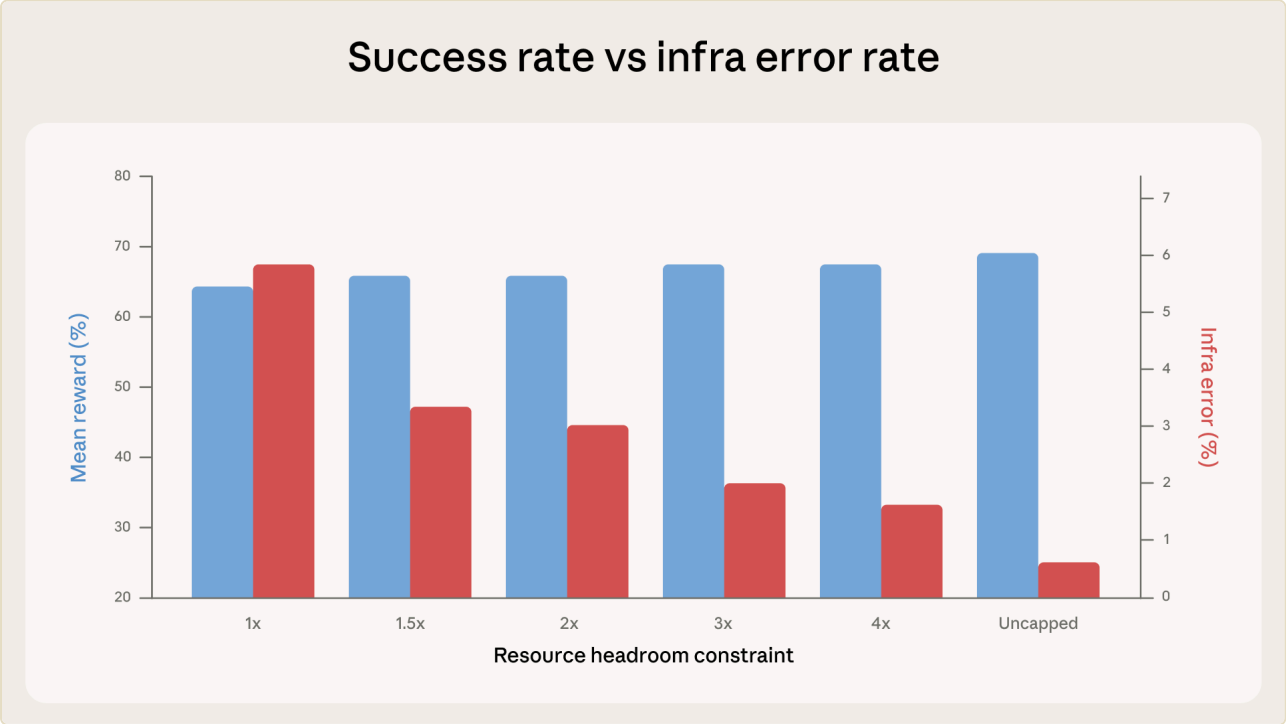
しかし 3x を超えると、追加リソースはエージェントが以前は解けなかった問題を解くのを積極的に助け始めます。これは、上限が eval の実際の測定対象を変えうることを示しています。厳しい上限は非常に効率的な戦略を偶然優遇し、寛大な上限は利用可能なリソースをより活用できるエージェントに有利に働きます。

無駄のない効率的なコードを素早く書くエージェントは厳しい制約下でうまくいきます。重量級ツールで解を総当たりするエージェントは寛大な環境でうまくいきます。どちらも正当なテスト対象ですが、リソース設定を指定せずに単一のスコアに集約すると、差異——そして現実世界への汎化性——を解釈することが難しくなります。

Terminal-Bench のタスクの一つ `bn-fit-modify`（ベイジアンネットワークのフィッティングを要求するタスク）では、モデルによっては最初の行動として標準的な Python データサイエンススタック（`pandas`、`networkx`、`scikit-learn` とそのツールチェーン全体）をインストールしようとします。寛大な上限ではこれが機能します。厳しい上限では、エージェントが解のコードを一行も書く前に、インストール中にポッドがメモリ不足になります。より無駄のない戦略（標準ライブラリのみを使って数学をゼロから実装する）も存在し、一部のモデルはそれをデフォルトとします。他のモデルはそうしません。モデルによってデフォルトのアプローチは異なり、リソース設定がどのアプローチが成功するかを決定します。

この主要な発見を、異なる Anthropic モデル間でも再現しました。効果の方向は一致していましたが、大きさは異なりました。同様のトレンドは Claude 以外のモデルでも成立すると思われますが、厳密には検証していません。

このパターンが Terminal-Bench 以外の eval でも成立するかどうかを検証するため、SWE-bench でクロスオーバー実験を実施しました。227 問題を各 10 サンプルで、ベースラインから最大 5x の合計 RAM を変動させました。同じ効果が確認されましたが、大きさは小さいものでした。スコアは再び RAM とともに単調に増加しましたが、5x では 1x より 1.54 パーセントポイントしか高くありませんでした。SWE-bench タスクはリソース集約度が低いため、より小さい効果が期待されますが、リソース割り当てがそこでも中立でないことを示しています。



その他の分散源

リソース割り当てだけが隠れた変数ではありません。特定の設定では、時間制限も役割を果たし始めます。

原則として、クラスターの健全性からハードウェア仕様まで、並列実行レベルから出力帯域幅まで、評価セットアップのあらゆる要素が最終スコアに影響を与えうります。エージェント型 eval は構造上エンドツーエンドのシステムテストであり、そのシステムのあらゆる構成要素が交絡因子として機能しえます。例えば、合格率が時間帯によって変動することを逸話的に観察しています。おそらくトラフィックパターンやインシデントによって API レイテンシが変化するためです。この効果を正式に定量化はしていませんが、より大きな点を示しています。「モデルの能力」と「インフラの挙動」の境界は、単一のベンチマークスコアが示唆するよりもはるかに曖昧なのです。モデルプロバイダーは専用ハードウェアによってこれから eval インフラを保護できますが、外部の評価者には同じことが容易にはできません。

公開ベンチマークは通常、純粋なモデルの能力を測定することを目的としていますが、実際にはインフラの特性と混同するリスクがあります。これが望ましい場合もあります。スタック全体のエンドツーエンドテストが可能になるからです。しかし多くの場合そうではありません。公開共有を目的としたコーディング eval では、複数回かつ複数日にわたって実行することで、ノイズを平均化するのに役立ちます。

推奨事項

理想的なシナリオは、eval を実行するスキャフォールドと推論スタックの両方を、まったく同じハードウェア条件で実行することです——これにより完全な再現性が保証されます。ただし、常に実用的とは限りません。

コンテナランタイムが実際にリソースを強制する方法（保証済み割り当てと別のハード kill 閾値）を考慮すると、eval はタスクごとに単一の固定値ではなく、両方のパラメーターを指定することを推奨します。単一の正確な仕様は保証済み割り当てを kill 閾値と等しく設定し、余裕をゼロにします。1x で文書化した一時的なメモリスパイクは、eval を不安定にするのに十分です。2 つのパラメーターを分離することで、偶発的な OOM kill を避けるための十分な余裕をコンテナに与えながら、スコア膨張を防ぐハード上限も維持できます。

その間の帯域は、フロアと上限のスコアがお互いのノイズの範囲内に収まるように調整する必要があります。例えば Terminal-Bench 2.0 では、タスクごとの仕様の 3x の上限がインフラのエラー率を約 3 分の 2 削減し（5.8% から 2.1%、$p < 0.001$）、スコアの向上を小さくノイズの範囲内に抑えました（$p = 0.40$）。これは合理的なトレードオフです。インフラの交絡因子はほぼ中和され、意味のあるリソース圧力が除去されません。正確な倍率はベンチマークとタスク分布によって異なるため報告されるべきですが、経験的な調整の原則は一般的です。

この研究の意義

これらの発見には、eval インフラを超えた実際的な影響があります。ベンチマークスコアは意思決定のインプットとして使われることが増えていますが、この注目（と依存）の増加は、それらがどのように実行・報告されるかの厳密さの向上と必ずしも伴っていません。現状では、リーダーボードでの 2 ポイントの差は、本物の能力差を反映している場合もあれば、一方の eval がより強力なハードウェアで、あるいはより幸運な時間帯に実行されたことを反映している場合もあります。公開（または標準化）されたセットアップ設定がなければ、関係者が同一条件下での結果を再現するという追加努力を行わない限り、外部から見分けることは困難です。

Anthropic のようなラボにとって、エージェント型 eval のリソース設定は、プロンプト形式やサンプリング温度と同じ厳密さで文書化・管理された第一級の実験変数として扱われるべきです。ベンチマークのメンテナーにとっては、推奨リソース仕様を公開すること（Terminal-Bench 2.0 が行っているように）が大きな助けになりますが、強制の方法論を指定することで私たちが発見したギャップを埋められるでしょう。そして、ベンチマーク結果を使用するすべての人にとっての核心的なポイントは、エージェント型 eval における小さなスコア差は、報告された数値の精度が示唆するよりも多くの不確実性を持つということです——特に一部の交絡因子は制御することが困難すぎるからです。

リソース方法論が標準化されるまで、私たちのデータは、エージェント型 eval での 3 パーセントポイント未満のリーダーボード差は、eval の設定が文書化されマッチングされるまで懐疑的に見るべきだと示唆しています。Terminal-Bench でのリソース設定の中程度の範囲における観測された差異は 2 パーセントポイント弱です。素朴な二項信頼区間はすでに 1〜2 パーセントポイントにわたります。ここで文書化したインフラの交絡因子はその上に積み重なり、その中にあるわけではありません。割り当て範囲の極端な領域では、差異は 6 に達します。

数ポイントのリードは本物の能力差を示しているかもしれません——あるいは単に大きな VM なだけかもしれません。

謝辞

執筆は Gian Segato。Nicholas Carlini、Jeremy Hadfield、Mike Merrill、Alex Shaw の貢献に特別に感謝します。この研究はコーディングエージェントの評価に取り組む複数のチームの共同努力を反映しています。貢献に興味のある候補者は anthropic.com/careers で応募を歓迎します。